

Breaking the Gómez-Torrecillas–Lobillo–Navarro Cryptosystem: LLL Recovers the Secret Key Within Seconds

Abul Kalam and Santanu Sarkar

Indian Institute of Technology Madras, India

`abulkalam.sunny@gmail.com`, `sarkar.santanu.bir1@gmail.com`

Abstract. We present a key-recovery attack on the knapsack McEliece-based public key cryptosystem recently proposed by Gómez-Torrecillas, Lobillo, and Navarro in DCC 2026. Given the public key alone, the attack recovers the complete private key in time polynomial in the bit-length of the public key, and therefore constitutes a total break. The attack combines two observations based solely on the public key. An integer combination of the public entries whose coefficients sum to zero eliminates the secret mask. If such a combination is also orthogonal to the public key and sufficiently short, then its coordinates are divisible by the corresponding private key integers. The modulus required for correct decryption creates a large norm gap between these short vectors satisfying the hidden private key relations and the remaining vectors in the lattice. LLL reduction exploits this gap to recover the short vectors and hence the corresponding divisibility relations. The private key integers and the remaining secret parameters are then recovered by combining information from multiple overlapping lattices and using greatest common divisors. We implemented the attack in SageMath and recovered the complete private key for all twenty four parameter sets proposed by the authors within 3 seconds, despite claimed security levels ranging from 64 to 156 bits. Our analysis further shows that the weakness is structural rather than a consequence of the particular parameter choices.

Keywords: public key cryptanalysis · knapsack cryptosystems · lattice basis reduction · LLL · orthogonal lattices

1 Introduction

Knapsack cryptosystems are among the earliest proposals for public key cryptography. Merkle and Hellman [MH78] showed that a superincreasing sequence makes the subset-sum problem efficiently solvable and proposed disguising such a sequence through modular multiplication to obtain a hard instance. The resulting construction was attractive for its efficiency, as both encryption and decryption require only additions and a modular multiplication, offering a substantial computational advantage over factoring-based alternatives such as RSA [RSA78]. This apparent efficiency, however, came at the expense of security. Shamir [Sha82]

showed that the basic construction could be broken in polynomial time by recovering an equivalent private key, while Brickell [Bri84] extended the cryptanalysis to iterated variants. These attacks revealed a broader structural weakness, namely that the superincreasing structure leaves information that is not completely hidden by the modular transformation. Odlyzko’s survey [Odl90] documents the history of knapsack cryptosystems and the cryptanalytic attacks that followed many of their proposals.

Lattice basis reduction has played a central role in the cryptanalysis of knapsack cryptosystems. The LLL algorithm [LLL82] finds relatively short vectors in a lattice in polynomial time, and hidden structure in a knapsack instance can often be represented by unusually short vectors in a lattice constructed from public information. Lagarias and Odlyzko [LO85] exploited this approach to solve low-density subset-sum instances, while Coster *et al.* [CJL⁺92] extended the range of densities for which lattice-based attacks are effective. Nguyen and Stern [NS97, NS99] introduced a complementary construction based on the orthogonal lattice, which transforms linear relations satisfied by the underlying secret data into short vectors of a lattice that can be constructed from public information. Coron and Gini [CG20] later showed that the original orthogonal-lattice algorithm for the hidden subset-sum problem has exponential running time and proposed a modified reduction step that yields a polynomial-time algorithm for substantially larger instances. More recently, Das and Joux [DJ24] presented a key-recovery attack on the partial Vandermonde knapsack problem that breaks parameter sets previously claimed to be secure.

Gómez-Torrecillas, Lobillo, and Navarro [GTLN26] recently proposed a knapsack cryptosystem in the McEliece [McE78] style. The private key consists of n pairwise coprime integers $p_1, \dots, p_n$ chosen from the interval $[2^b, 2^{b+\beta})$, together with a modulus g coprime to each p_i and a mask u . The public key is the vector whose entries are $t_i \equiv p_i^{-1} - u \pmod{g}$. A plaintext is a weight- t vector $\mathbf{e}$ whose nonzero entries belong to $\{1, \dots, 2^b - 1\}$. Thus, unlike the binary subset-sum setting, the plaintext is a low-weight error pattern in the spirit of McEliece. The ciphertext is $(c_1, c_2) = (\sum_i e_i t_i, \sum_i e_i)$, where both sums are taken over the integers. The second component allows the receiver to remove the mask, since $c_1 + uc_2$ reduces modulo g to $\sum_i e_i p_i^{-1}$.

Decryption does not rely on recovering a superincreasing sequence. After removing the mask, the receiver performs rational reconstruction on the resulting residue using a truncated extended Euclidean algorithm. The denominator of the reconstructed fraction is the product of the private key integers corresponding to the nonzero positions of the plaintext. The message support can therefore be identified by testing which p_i divide this denominator, after which the corresponding coefficients are recovered by modular inversion. For the rational reconstruction to be unique, g must be sufficiently large compared with the square of the largest product of t private key integers. Since each p_i has at most $b + \beta$ bits, this requires $\log_2 g$ to be on the order of $2t(b + \beta)$. The authors propose twenty-four parameter sets with n ranging from 85 to 400 and claimed security levels ranging from

64 to 156 bits. As we show in this work, the modulus size required for correct decryption also creates the separation exploited by our lattice attack.

1.1 Our Contribution

In this work, we show that the knapsack McEliece-based cryptosystem of [GTLN26] is insecure. Our main result is a key-recovery attack that, given only the public key $(t_1, \dots, t_n)$, recovers the complete private key $(p_1, \dots, p_n, g, u)$ in time polynomial in the bit-length of the public key. Thus, the attack constitutes a total break of the cryptosystem. The attack is also practical. Our SageMath implementation recovers the complete private key for all twenty-four parameter sets proposed in [GTLN26] within 3 seconds on a single CPU core, despite claimed security levels ranging from 64 to 156 bits.

On the technical side, the attack is based on two linear conditions that can be imposed using the public key alone. For an integer vector $\mathbf{c}$ whose coordinates sum to zero, the mask u is eliminated from the corresponding linear combination of the public entries. If, in addition, $\mathbf{c}$ is orthogonal to the public key vector, then $\mathbf{c} \cdot \boldsymbol{\pi} \equiv 0 \pmod{g}$, where $\boldsymbol{\pi} = (P_1, \dots, P_m)$ is the vector of complementary products of the private key integers for the considered block. For sufficiently short vectors, this congruence lifts to the exact relation $\mathbf{c} \cdot \boldsymbol{\pi} = 0$, which in turn implies $p_k \mid c_k$ for every coordinate k . We therefore define the publicly computable lattice $\mathcal{L}_0$ by the two public linear conditions, while the vectors satisfying the additional exact relation $\mathbf{c} \cdot \boldsymbol{\pi} = 0$ form the sublattice $\mathcal{L}_1$. We construct these lattices on blocks of m coordinates at a time, keeping their dimension small. The effectiveness of the construction comes from the size of the modulus. Correct decryption requires $\log_2 g$ to be of order $2t(b + \beta)$, whereas each p_i has only $b + \beta$ bits. This creates a substantial separation between the short vectors in $\mathcal{L}_1$ and the remaining vectors in $\mathcal{L}_0$, allowing LLL to recover vectors that reveal divisibility information about the private key integers.

The attack proceeds by first extracting partial information about the private key integers from several overlapping blocks. A single block provides divisibility relations that determine each shared p_k only up to an unknown multiplicative factor. To eliminate this ambiguity, we construct several blocks that contain the same set of κ coordinates and combine the resulting relations by taking their greatest common divisors. For each shared coordinate k , this yields a value of the form $\gamma_k = f_k p_k$, where f_k is a small cofactor. Since the private key integers are known to lie in the public interval $[2^b, 2^{b+\beta})$, the correct multiple can be identified by testing the possible quotients against this interval.

Once sufficiently many p_k are known, the modulus g is recovered from the public key relation $g \mid p_i p_j (t_j - t_i) + (p_i - p_j)$. Since p_i and p_j are known, the right-hand side is a known multiple of g . As with the recovery of the p_k , the remaining ambiguity is only a small cofactor, which can be removed using the public bounds on g . The mask u and the remaining private key integers are then recovered directly from the public key equations using modular inversions. In particular, this final step also recovers the coordinates that were not included in any lattice block.

An important feature of the attack is that its analysis does not require the private key integers p_i to be prime. Pairwise coprimality is sufficient for the divisibility argument. Consequently, replacing the prime private key integers by composite pairwise coprime integers does not remove the underlying weakness or affect the attack.

Source Code. The implementation of the attack is publicly available at <https://github.com/Abulkalam-sunny/Breaking-the-Gomez-Torrecillas--Lobillo--Navarro-Cryptosystem.git>.

Outline. The remainder of the paper is organized as follows. Section 2 introduces the required preliminaries and Section 3 describes the Gómez-Torrecillas–Lobillo–Navarro cryptosystem. Section 4 establishes the structural properties underlying the attack. Section 5 develops the lattice-based recovery method, and Section 6 presents the complete attack. Section 7 evaluates the attack on the proposed parameter sets. Finally, Section 8 concludes the paper.

2 Preliminaries

In this section, we introduce the notation and basic definitions used throughout the paper. Bold uppercase letters, such as $\mathbf{M}$, denote matrices, whereas bold lowercase letters, such as $\mathbf{x}$, denote vectors. For vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^m$, their Euclidean inner product is denoted by $\mathbf{x} \cdot \mathbf{y}$. The Euclidean norm of $\mathbf{x}$ is denoted by $\|\mathbf{x}\|_2$ and its ℓ_∞ -norm is defined by $\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq m} |x_i|$. For a positive integer n , we write $[n] = \{1, 2, \dots, n\}$.

2.1 Lattices

We first recall the basic definitions and properties of lattices that will be used throughout the paper.

Definition 1 (Lattice). Let $\mathbf{B} \in \mathbb{R}^{m \times n}$ have linearly independent columns $\mathbf{b}_1, \dots, \mathbf{b}_n \in \mathbb{R}^m$. The lattice generated by $\mathbf{B}$ is

$$\mathcal{L}(\mathbf{B}) = \{\mathbf{B}\mathbf{x} : \mathbf{x} \in \mathbb{Z}^n\} = \left\{ \sum_{i=1}^n x_i \mathbf{b}_i : x_i \in \mathbb{Z} \right\}.$$

The vectors $\mathbf{b}_1, \dots, \mathbf{b}_n$ form a basis of $\mathcal{L}(\mathbf{B})$.

The integer n is called the *rank* of the lattice, and m is its *ambient dimension*. A lattice whose rank equals its ambient dimension is called *full-rank*. The determinant, also called the *covolume*, of a rank- n lattice $\mathcal{L}(\mathbf{B})$ is defined as $\det(\mathcal{L}(\mathbf{B})) = \sqrt{\det(\mathbf{B}^\top \mathbf{B})}$. Equivalently, $\det(\mathcal{L})$ is the n -dimensional volume of a fundamental parallelepiped of $\mathcal{L}$. In the full-rank case $n = m$, this reduces to $\det(\mathcal{L}) = |\det(\mathbf{B})|$.

The determinant is an intrinsic property of the lattice and is independent of the choice of basis. Indeed, if $\mathbf{U} \in \text{GL}_n(\mathbb{Z})$ is a unimodular matrix and $\mathbf{B}' = \mathbf{B}\mathbf{U}$, then

$$\mathcal{L}(\mathbf{B}') = \mathcal{L}(\mathbf{B}), \quad \text{and} \quad \det(\mathcal{L}(\mathbf{B}')) = \det(\mathcal{L}(\mathbf{B})).$$

The determinant can be bounded in terms of the lengths of the basis vectors by Hadamard's inequality.

Theorem 1 (Hadamard's inequality). *Let $\mathbf{b}_1, \dots, \mathbf{b}_n \in \mathbb{R}^m$ be linearly independent, and let $\mathbf{B} = [\mathbf{b}_1 \mid \dots \mid \mathbf{b}_n]$. Then*

$$\det(\mathcal{L}(\mathbf{B})) \leq \prod_{i=1}^n \|\mathbf{b}_i\|_2.$$

Equality holds if and only if $\mathbf{b}_1, \dots, \mathbf{b}_n$ are pairwise orthogonal.

For a lattice contained in $\mathbb{Z}^m$, we consider the integer points that are orthogonal to every vector in the lattice.

Definition 2 (Integer orthogonal lattice). *Let $\mathcal{L} \subseteq \mathbb{Z}^m$ be a lattice. Its integer orthogonal lattice is defined as*

$$\mathcal{L}^\perp = \{\mathbf{y} \in \mathbb{Z}^m : \mathbf{x} \cdot \mathbf{y} = 0 \text{ for all } \mathbf{x} \in \mathcal{L}\}.$$

We will use the following standard consequence of the orthogonal lattice theorem.

Theorem 2 ([NS99]). *Let $\mathcal{L} \subseteq \mathbb{Z}^m$ be a lattice, and let $\bar{\mathcal{L}} = (\mathcal{L}^\perp)^\perp$. Then*

$$\text{rank}(\mathcal{L}) + \text{rank}(\mathcal{L}^\perp) = m, \quad \text{and} \quad \det(\mathcal{L}^\perp) = \det(\bar{\mathcal{L}}).$$

2.2 Successive Minima

We next introduce the successive minima, which quantify the lengths required to obtain linearly independent lattice vectors.

Definition 3 (Successive Minima). *Let $\mathcal{L} \subseteq \mathbb{R}^m$ be a rank- n lattice. For $i \in [n]$, the i -th successive minimum of $\mathcal{L}$ is defined as*

$$\lambda_i(\mathcal{L}) = \min \left\{ r > 0 : \begin{array}{l} \text{there exist linearly independent vectors } \mathbf{v}_1, \dots, \mathbf{v}_i \in \mathcal{L} \\ \text{such that } \|\mathbf{v}_j\|_2 \leq r \text{ for all } j \in [i] \end{array} \right\}.$$

The successive minima satisfy $0 < \lambda_1(\mathcal{L}) \leq \lambda_2(\mathcal{L}) \leq \dots \leq \lambda_n(\mathcal{L})$. In particular, $\lambda_1(\mathcal{L})$ is the length of a shortest nonzero vector in $\mathcal{L}$.

Let $B_2^n = \{\mathbf{x} \in \text{span}_{\mathbb{R}}(\mathcal{L}) : \|\mathbf{x}\|_2 \leq 1\}$ denote the Euclidean unit ball in the n -dimensional linear span of $\mathcal{L}$. Its n -dimensional volume is $\text{vol}(B_2^n) = \frac{\pi^{n/2}}{\Gamma(n/2+1)}$.

Theorem 3 (Minkowski's Second Theorem [CC71]). *Let $\mathcal{L} \subseteq \mathbb{R}^m$ be a rank- n lattice. Then*

$$\frac{2^n}{n!} \det(\mathcal{L}) \leq \text{vol}(B_2^n) \prod_{i=1}^n \lambda_i(\mathcal{L}) \leq 2^n \det(\mathcal{L}).$$

Equivalently,

$$\frac{2^n}{n! \text{vol}(B_2^n)} \det(\mathcal{L}) \leq \prod_{i=1}^n \lambda_i(\mathcal{L}) \leq \frac{2^n}{\text{vol}(B_2^n)} \det(\mathcal{L}).$$

2.3 LLL-Reduced Bases

Lattice reduction algorithms transform a given lattice basis into a basis consisting of relatively short and nearly orthogonal vectors. Among the most widely used lattice reduction algorithms is the Lenstra–Lenstra–Lovász (LLL) algorithm [LLL82].

Let $\mathbf{B} = (\mathbf{b}_1, \dots, \mathbf{b}_n)$ be a basis of a rank- n lattice $\mathcal{L}$. The *Gram–Schmidt orthogonalization* (GSO) of $\mathbf{B}$ is the sequence $\mathbf{b}_1^*, \dots, \mathbf{b}_n^*$ defined recursively by

$$\mathbf{b}_i^* = \mathbf{b}_i - \sum_{j=1}^{i-1} \mu_{i,j} \mathbf{b}_j^*, \quad \mu_{i,j} = \frac{\langle \mathbf{b}_i, \mathbf{b}_j^* \rangle}{\|\mathbf{b}_j^*\|_2^2}.$$

Definition 4 (LLL-reduced basis). *Let $\delta \in (1/4, 1]$. A basis $\mathbf{B} = (\mathbf{b}_1, \dots, \mathbf{b}_n)$ is called δ -LLL-reduced if it satisfies the following conditions:*

1. *Size reduction:* $|\mu_{i,j}| \leq \frac{1}{2}$, for $1 \leq j < i \leq n$.
2. *Lovász condition:* $\delta \|\mathbf{b}_{i-1}^*\|_2^2 \leq \|\mathbf{b}_i^*\|_2^2 + \mu_{i,i-1}^2 \|\mathbf{b}_{i-1}^*\|_2^2$, for $2 \leq i \leq n$.

A commonly used choice is $\delta = 3/4$. Unless otherwise stated, an *LLL-reduced basis* refers to a δ -LLL-reduced basis with $\delta = 3/4$.

The following standard bounds relate the lengths of an LLL-reduced basis to the successive minima of the underlying lattice.

Proposition 1 ([LLL82]). *Let $\mathbf{b}_1, \dots, \mathbf{b}_n$ be a LLL-reduced basis of a lattice $\mathcal{L}$ of rank n . Let $\mathbf{x}_1, \dots, \mathbf{x}_s \in \mathcal{L}$ be linearly independent. Then, for every $1 \leq j \leq s$,*

$$\|\mathbf{b}_j\|_2 \leq 2^{(n-1)/2} \max_{1 \leq i \leq s} \|\mathbf{x}_i\|_2.$$

Equivalently, in terms of the successive minima of $\mathcal{L}$,

$$\|\mathbf{b}_j\|_2 \leq 2^{(n-1)/2} \lambda_i(\mathcal{L}), \quad 1 \leq j \leq i \leq n.$$

3 Knapsack McEliece-Based GTLN Cryptosystem

Gómez-Torrecillas, Lobillo, and Navarro [GTLN26] proposed a Knapsack McEliece-based public key encryption (PKE) scheme together with a corresponding key encapsulation mechanism (KEM). The key-generation procedure is identical for both constructions. We briefly recall the cryptosystem, restricting our description to the components relevant to our cryptanalysis, with particular emphasis on the key-generation procedure.

The scheme is parameterized by the plaintext length n , the plaintext weight t , the alphabet size $z = 2^b$, and a parameter β controlling the size of the secret integers. The private key consists of a collection of pairwise coprime integers $p_1, \dots, p_n$, a secret modulus g , and an additive mask u . The public key is constructed from shifted modular inverses of the secret integers.

We first describe the key-generation procedure and then derive the constraint on the modulus g imposed by the correctness of decryption, following the concrete realization in Section 5 of [GTLN26].

Key Generation: The key-generation procedure consists of the following steps.

1. Set $D = b + \beta$ and choose n pairwise coprime integers $p_1, \dots, p_n$ satisfying

$$2^b \leq p_i < 2^D, \quad i \in [n]. \quad (1)$$

In the concrete realization of [GTLN26, Section 5], the integers $p_1, \dots, p_n$ are chosen to be prime.

2. Define $B = \max \{b + \lceil \log_2 t \rceil + (t-1)D, tD\}$, and choose an integer modulus g of bit-length $2B + 3$ such that $\gcd(g, p_i) = 1$, for $i \in [n]$.
3. For each $i \in [n]$, compute $h_i \equiv p_i^{-1} \pmod{g}$. Choose $u \in \{0, \dots, g-1\}$ uniformly at random subject to $u \neq h_i$, for $i \in [n]$.
4. Compute the public values $t_i \equiv (h_i - u) \pmod{g}$, for $i \in [n]$, or, equivalently,

$$t_i \equiv p_i^{-1} - u \pmod{g}. \quad (2)$$

The resulting private and public keys are, respectively,

$$\text{sk} = (p_1, \dots, p_n, g, u) \quad \text{and} \quad \text{pk} = (t_1, \dots, t_n).$$

Encryption and Decryption: A plaintext is represented by a vector $\mathbf{e} = (e_1, \dots, e_n) \in \{0, \dots, z-1\}^n$ with exactly t nonzero entries. The encryption algorithm computes

$$c_1 = \sum_{i=1}^n e_i t_i, \quad \text{and} \quad c_2 = \sum_{i=1}^n e_i$$

over the integers. During decryption, the receiver computes $s \equiv (c_1 + u c_2) \pmod{g}$. By the definition of the public key components in (2),

$$s \equiv \sum_{i=1}^n e_i (t_i + u) \equiv \sum_{i=1}^n e_i h_i \pmod{g}.$$

The decryption algorithm then recovers the support and the corresponding plaintext entries using a truncated extended Euclidean algorithm followed by congruences modulo the individual p_i . For the detailed decryption algorithm, we refer the reader to [GTLN26]. For our purposes, these details are not needed; we only require the constraint on g imposed by the correctness of decryption.

The Constraint Imposed by Correctness: Let $\lambda_{\max} = p_{n_1} \cdots p_{n_t}$ denote the product of the t largest integers in $\{p_1, \dots, p_n\}$. Theorem 7 of [GTLN26] establishes that the decryption procedure returns the correct plaintext provided

$$g > 4\lambda_{\max}^2. \quad (3)$$

Since every $p_i \geq 2^b$, we have $\lambda_{\max} \geq 2^{tb}$. Thus, (3) imposes the following lower bound on the bit-length $G = \lceil \log_2 g \rceil$:

$$G \geq 2tb + 3. \quad (4)$$

The concrete realization in [GTLN26, Section 5] uses $G = 2B + 3$ with $B \geq tD$, and therefore satisfies the stronger bound $G \geq 2tD + 3$. In our analysis, however, we use only the weaker bound (4). This ensures that our results apply to any correct instantiation of the cryptosystem, rather than being restricted to the specific parameter sets proposed in [GTLN26].

This correctness requirement creates a substantial gap between the sizes of the secret integers $\{p_1, \dots, p_n\}$ and the modulus g . For the smallest recommended parameter set, each p_i have 61 bits, whereas g has at least 2643 bits; for the largest parameter set, the corresponding sizes are 51 bits and at least 5003 bits, respectively. Thus, the correctness requirement forces each p_i to be exponentially smaller than the modulus g . It is precisely this gap that underlies the attack developed in the subsequent sections.

Parameter Sets: The proposed parameter sets and their corresponding claimed security levels are summarized in Table 1. The claimed security level is the minimum estimated complexity, in bits, among the attacks considered in [GTLN26].

4 Structural Properties of the public key

Fix a set S of $m(\leq n)$ indices and, for convenience, relabel its elements as $1, \dots, m$. Define

$$Q = \prod_{i \in S} p_i, \quad P_i = \frac{Q}{p_i} = \prod_{\substack{j \in S \\ j \neq i}} p_j, \quad \pi = (P_1, \dots, P_m) \in \mathbb{Z}^m,$$

and let $\mathbf{t} = (t_1, \dots, t_m)$ denote the corresponding entries of the public key. Since $p_i < 2^D$ for every $i \in S$, each P_i satisfies

$$0 < P_i < 2^{(m-1)D}. \quad (5)$$

Table 1: Claimed security levels of the proposed parameter sets in [GTLN26].

n	t	b	β	Claimed Security (bits)	n	t	b	β	Claimed Security (bits)
85	22	60	1	74	90	23	60	1	76
90	26	50	15	65	95	22	70	1	79
95	25	55	1	78	100	20	50	1	78
100	25	55	1	80	110	25	60	1	84
110	30	40	20	64	125	20	60	1	87
125	25	40	1	87	150	20	40	5	87
150	30	30	20	73	150	30	50	1	98
175	30	45	1	105	175	40	30	20	73
200	40	70	1	117	200	50	45	10	99
225	45	45	5	118	225	55	40	10	103
250	60	50	1	132	250	55	30	20	85
300	60	50	1	150	400	50	50	1	156

The vector $\boldsymbol{\pi}$ is not known to the attacker and is introduced only for the analysis. The attack instead searches for integer vectors $\mathbf{c} \in \mathbb{Z}^m$ satisfying the two publicly computable linear conditions $\mathbf{c} \cdot \mathbf{1} = 0$ and $\mathbf{c} \cdot \mathbf{t} = 0$. The following lemmas show that such vectors reveal the hidden structure determined by the secret integers p_i .

Lemma 1. *Let $\mathbf{c} \in \mathbb{Z}^m$ satisfy $\mathbf{c} \cdot \mathbf{1} = 0$ and $\mathbf{c} \cdot \mathbf{t} = 0$. Then $\mathbf{c} \cdot \boldsymbol{\pi} \equiv 0 \pmod{g}$. Moreover, if*

$$\|\mathbf{c}\|_\infty < \frac{g}{m2^{(m-1)D}}, \quad (6)$$

then $\mathbf{c} \cdot \boldsymbol{\pi} = 0$ holds in integer.

Proof. From the definition of the public key entries in (2), $t_i \equiv h_i - u \pmod{g}$. Since $\mathbf{c} \cdot \mathbf{1} = 0$ and $\mathbf{c} \cdot \mathbf{t} = 0$, we have

$$\sum_{i=1}^m c_i h_i \equiv \sum_{i=1}^m c_i (t_i + u) \equiv 0 \pmod{g}.$$

Multiplying by Q and using $Qh_i = P_i(p_i h_i) \equiv P_i \pmod{g}$, gives

$$\mathbf{c} \cdot \boldsymbol{\pi} = \sum_{i=1}^m c_i P_i \equiv 0 \pmod{g},$$

where we have used $\gcd(Q, g) = 1$.

It remains to show that the congruence is an equality for sufficiently short $\mathbf{c}$. By (5), $0 < P_i < 2^{(m-1)D}$ for every $i \in [m]$. Hence

$$|\mathbf{c} \cdot \boldsymbol{\pi}| \leq \sum_{i=1}^m |c_i| P_i < m \|\mathbf{c}\|_\infty 2^{(m-1)D}.$$

Under (6), the right-hand side is strictly smaller than g . Since $\mathbf{c} \cdot \boldsymbol{\pi} \equiv 0 \pmod{g}$, it follows that $\mathbf{c} \cdot \boldsymbol{\pi} = 0$.

Lemma 1 shows that the common additive mask u can be eliminated by imposing the two linear conditions $\mathbf{c} \cdot \mathbf{1} = 0$ and $\mathbf{c} \cdot \mathbf{t} = 0$. This is particularly relevant since the purpose of u in [GTLN26, Section 4.1] is to obscure the relation between the public key entries and the secret integers. However, the lemma shows that this masking does not persist under those linear combinations of the public key entries.

One might instead consider using a distinct mask u_i for each coordinate to prevent such cancellation. This modification is incompatible with the decryption mechanism, which relies on the common mask u to recover $\sum_i e_i h_i$ from the single combination $c_1 + uc_2$.

Remark 1. The contrapositive of Lemma 1 shows that any vector $\mathbf{c}$ satisfying $\mathbf{c} \cdot \mathbf{1} = 0$ and $\mathbf{c} \cdot \mathbf{t} = 0$, but not satisfying $\mathbf{c} \cdot \boldsymbol{\pi} = 0$, must satisfy $\|\mathbf{c}\|_\infty \geq g/(m2^{(m-1)D})$. Thus, the bound serves two purposes: as a guarantee that short vectors are useful and as a guarantee that useless vectors are necessarily long.

Lemma 2. *If $\mathbf{c} \in \mathbb{Z}^m$ satisfies $\mathbf{c} \cdot \boldsymbol{\pi} = 0$, then $p_k \mid c_k$ for every $k \in S$.*

Proof. Fix $k \in S$ and reduce $\mathbf{c} \cdot \boldsymbol{\pi} = \sum_{i=1}^m c_i P_i = 0$ modulo p_k . For every $i \neq k$, the integer P_i contains p_k as a factor, and hence $P_i \equiv 0 \pmod{p_k}$. It follows that $c_k P_k \equiv 0 \pmod{p_k}$. Pairwise coprimality of the p_j gives $\gcd(P_k, p_k) = 1$, and therefore $p_k \mid c_k$.

Observe that Lemma 2 requires only pairwise coprimality, rather than the stronger assumption of primality.

Remark 2. Taken together, Lemmas 1 and 2 show that key recovery can be reduced to finding sufficiently short integer vectors $\mathbf{c} \in \mathbb{Z}^m$ satisfying the two publicly computable conditions $\mathbf{c} \cdot \mathbf{1} = 0$ and $\mathbf{c} \cdot \mathbf{t} = 0$. For such short vectors, Lemma 1 yields the hidden relation $\mathbf{c} \cdot \boldsymbol{\pi} = 0$, while Lemma 2 then implies $p_k \mid c_k$ for every $k \in S$. Thus, sufficiently short vectors satisfying the two public linear conditions expose divisibility relations involving the secret integers p_k . It remains to show that a sufficient number of such vectors exist and can be recovered efficiently from the public key. We address this question using lattice reduction in the next section.

5 Lattice Structure of the Associated Integer Kernel

We now show that the short vectors identified in previous section can be recovered from the public key using lattice reduction. The key point is that the attacker can construct a lattice containing both useful and useless vectors, while Lemma 1 provides a lower bound on the norm of every useless vector.

Fix the set S of m indices considered in Section 4. Define

$$\mathcal{L}_0 = \{\mathbf{c} \in \mathbb{Z}^m : \mathbf{c} \cdot \mathbf{1} = 0, \mathbf{c} \cdot \mathbf{t} = 0\}, \quad \text{and} \quad \mathcal{L}_1 = \{\mathbf{c} \in \mathcal{L}_0 : \mathbf{c} \cdot \boldsymbol{\pi} = 0\}.$$

The lattice $\mathcal{L}_0$ is completely determined by the public key and can therefore be computed by the attacker. In contrast, $\mathcal{L}_1$ depends on the unknown private key integers through $\boldsymbol{\pi}$ and is not directly available.

Assume that $\{\mathbf{1}, \mathbf{t}, \boldsymbol{\pi}\}$ are linearly independent over $\mathbb{Q}$. This is the generic situation for the parameter choices considered in the scheme, where the private key integers p_j , $j \in [n]$, are chosen to be distinct. Under this assumption,

$$\text{rank}(\mathcal{L}_0) = m - 2, \quad \text{rank}(\mathcal{L}_1) = m - 3.$$

By Lemma 2, every vector $\mathbf{c}$ in $\mathcal{L}_1$ yields divisibility information about the private key integers, that is, $p_k \mid c_k$ for every $k \in S$. The attack therefore amounts to finding sufficiently short vectors in $\mathcal{L}_1$ while avoiding vectors in $\mathcal{L}_0 \setminus \mathcal{L}_1$. We therefore characterize the structure of $\mathcal{L}_1$ in the following.

From the definition of the public key entries in (2), we have $g \mid p_i t_i + p_i u - 1$ for every $i \in S$. Define

$$k_i = \frac{p_i t_i + p_i u - 1}{g}, \quad \mathbf{k} = (k_1, k_2, \dots, k_m).$$

Therefore, we have

$$p_i t_i = 1 - p_i u + g k_i. \quad (7)$$

Since $0 \leq t_i, u < g$, we obtain $0 \leq p_i t_i + p_i u - 1 < 2p_i g$, and thus $0 \leq k_i < 2p_i$. Using (1), we consequently obtain

$$0 \leq k_i < 2^{D+1}. \quad (8)$$

Theorem 4. Let $\Delta = \text{diag}(p_1, \dots, p_m)$, $\mathbf{p} = (p_1, \dots, p_m)$, and define

$$\mathcal{K} = \{\mathbf{c}' \in \mathbb{Z}^m : \mathbf{c}' \cdot \mathbf{1} = 0, \mathbf{c}' \cdot \mathbf{p} = 0, \mathbf{c}' \cdot \mathbf{k} = 0\}, \quad \text{and} \quad \Delta\mathcal{K} = \{\Delta\mathbf{c}' : \mathbf{c}' \in \mathcal{K}\}$$

Then $\mathcal{L}_1 = \Delta\mathcal{K}$, and $\text{rank}(\mathcal{K}) = \text{rank}(\mathcal{L}_1) = m - 3$.

Proof. Let $\mathbf{c} \in \mathcal{L}_1$. By Lemma 2, $p_i \mid c_i$ for every $i \in S$, so $\mathbf{c} = \Delta\mathbf{c}'$ for a unique $\mathbf{c}' \in \mathbb{Z}^m$. The three defining conditions become

$$\begin{aligned} \mathbf{c} \cdot \mathbf{P} &= \sum_{i \in S} p_i c'_i \left(\frac{Q}{p_i} \right) = Q(\mathbf{c}' \cdot \mathbf{1}), \\ \mathbf{c} \cdot \mathbf{1} &= \sum_{i \in S} p_i c'_i = \mathbf{c}' \cdot \mathbf{p}, \\ \mathbf{c} \cdot \mathbf{t} &= \sum_{i \in S} c'_i (p_i t_i) \stackrel{(7)}{=} (\mathbf{c}' \cdot \mathbf{1}) - u(\mathbf{c}' \cdot \mathbf{p}) + g(\mathbf{c}' \cdot \mathbf{k}). \end{aligned}$$

Since $\mathbf{c} \in \mathcal{L}_1$, these three equations imply $\mathbf{c}' \cdot \mathbf{1} = \mathbf{c}' \cdot \mathbf{p} = \mathbf{c}' \cdot \mathbf{k} = 0$. Thus $\mathbf{c}' \in \mathcal{K}$, and hence $\mathcal{L}_1 \subseteq \Delta\mathcal{K}$. Conversely, if $\mathbf{c}' \in \mathcal{K}$, the same three identities, read in reverse, show that $\Delta\mathbf{c}' \in \mathcal{L}_1$. Therefore $\mathcal{L}_1 = \Delta\mathcal{K}$. Finally, Δ is invertible over $\mathbb{Q}$ and therefore preserves rank, giving $\text{rank}(\mathcal{K}) = \text{rank}(\mathcal{L}_1) = m - 3$.

Theorem 4 reveals the key structural feature of $\mathcal{L}_1$. Although the vectors in $\mathcal{L}_1$ are defined through the public quantities $\mathbf{t}$, which have size controlled by the large modulus g , the substitution $\mathbf{c} = \Delta\mathbf{c}'$ eliminates g completely. The resulting lattice $\mathcal{K}$ is defined only by the three vectors $\mathbf{1}$, $\mathbf{p}$, and $\mathbf{k}$. Moreover, the entries of $\mathbf{p}$ are bounded by 2^D using (1), while (8) gives $k_i < 2^{D+1}$. Thus, the determinant of $\mathcal{K}$ can be bounded in terms of quantities of size 2^{2D} and is independent of the size of g .

This factorization explains why $\mathcal{L}_1$ can contain vectors that are substantially shorter than those outside $\mathcal{L}_1$. In particular, the useful sublattice $\mathcal{L}_1$ inherits its short-vector structure from the much smaller lattice $\mathcal{K}$, whereas the separation bound for $\mathcal{L}_0 \setminus \mathcal{L}_1$ depends explicitly on the large modulus g . The next result makes this distinction quantitative by bounding the successive minima of $\mathcal{K}$ and, consequently, of $\mathcal{L}_1$.

Proposition 2. Put $r = m - 3$ and $\kappa(m) = \frac{2^{m-2}m^{3/2}}{\text{vol}(B_2^r)}$ with $\text{vol}(B_2^n) = \frac{\pi^{n/2}}{\Gamma(n/2+1)}$. Then

$$\lambda_r(\mathcal{K}) \leq \kappa(m)2^{2D}, \quad \text{and} \quad \lambda_r(\mathcal{L}_1) \leq \kappa(m)2^{3D}.$$

Proof. Let $\mathcal{M} = \mathbb{Z}\mathbf{1} + \mathbb{Z}\mathbf{p} + \mathbb{Z}\mathbf{k}$. Since $\mathcal{K} = \mathcal{M}^\perp$, Theorem 2 gives

$$\det(\mathcal{K}) = \det(\overline{\mathcal{M}}) \leq \det(\mathcal{M}), \quad \overline{\mathcal{M}} = (\mathcal{M}^\perp)^\perp,$$

where the inequality follows from $\mathcal{M} \subseteq \overline{\mathcal{M}}$. By Hadamard's inequality,

$$\det(\mathcal{M}) \leq \|\mathbf{1}\|_2 \|\mathbf{p}\|_2 \|\mathbf{k}\|_2 < \sqrt{m} \cdot \sqrt{m} 2^D \cdot \sqrt{m} 2^{D+1} = 2^{2D+1} m^{3/2},$$

where $\|\mathbf{p}\|_2 < \sqrt{m} 2^D$ follows from (1), and $\|\mathbf{k}\|_2 < \sqrt{m} 2^{D+1}$ follows from (8).

Since $\mathcal{K} \subseteq \mathbb{Z}^m$, we have $\lambda_j(\mathcal{K}) \geq 1$ for all j . Hence, by Minkowski's second theorem,

$$\lambda_r(\mathcal{K}) \leq \prod_{j=1}^r \lambda_j(\mathcal{K}) \leq \frac{2^r \det(\mathcal{K})}{\text{vol}(B_2^r)} < \frac{2^{r+1} m^{3/2}}{\text{vol}(B_2^r)} 2^{2D} = \kappa(m) 2^{2D}.$$

Finally, choose linearly independent $\mathbf{c}'_1, \dots, \mathbf{c}'_r \in \mathcal{K}$ with $\|\mathbf{c}'_j\|_2 \leq \lambda_r(\mathcal{K})$. By Theorem 4, $\Delta\mathbf{c}'_1, \dots, \Delta\mathbf{c}'_r \in \mathcal{L}_1$ are linearly independent. Since $p_i < 2^D$, we obtain $\|\Delta\mathbf{c}'_j\|_2 \leq 2^D \|\mathbf{c}'_j\|_2 \leq 2^D \lambda_r(\mathcal{K})$, for all $j \in [r]$ and therefore $\lambda_r(\mathcal{L}_1) \leq 2^D \lambda_r(\mathcal{K}) \leq \kappa(m) 2^{3D}$.

Proposition 2 shows that $\mathcal{L}_1$ contains $r = m - 3$ linearly independent vectors of sufficiently small norm. Recall that vectors in $\mathcal{L}_1$ provide the divisibility relations $p_k \mid c_k$ for $k \in S$ which are the key information required by the attack. Thus, recovering a sufficiently short set of linearly independent vectors from $\mathcal{L}_1$ is the central algorithmic objective. The next theorem shows that these vectors can be obtained by applying LLL reduction to the publicly computable lattice $\mathcal{L}_0$, provided that the short vectors in $\mathcal{L}_1$ are sufficiently separated from the vectors in $\mathcal{L}_0 \setminus \mathcal{L}_1$.

Theorem 5. Let $r = m - 3 \geq 1$, and let $\mathbf{b}_1, \dots, \mathbf{b}_{m-2}$ be an LLL-reduced basis of $\mathcal{L}_0$, with reduction parameter $\delta = 3/4$, sorted by increasing norm. If

$$2^{r/2} \kappa(m) 2^{3D} < \frac{g}{m 2^{(m-1)D}}, \quad (9)$$

then $\mathbf{b}_1, \dots, \mathbf{b}_r \in \mathcal{L}_1$, so that $p_k \mid (\mathbf{b}_j)_k$ for all $1 \leq j \leq r$, $k \in S$, while $\mathbf{b}_{m-2} \notin \mathcal{L}_1$.

Proof. Since $\mathcal{L}_1 \subseteq \mathcal{L}_0$, monotonicity of successive minima and Proposition 2 give $\lambda_r(\mathcal{L}_0) \leq \lambda_r(\mathcal{L}_1) \leq \kappa(m) 2^{3D}$. Applying Proposition 1 with $i = r$ to $\mathcal{L}_0$, whose rank is $r + 1$, yields $\|\mathbf{b}_j\|_2 \leq 2^{r/2} \lambda_r(\mathcal{L}_0) \leq 2^{r/2} \kappa(m) 2^{3D}$ for $j \leq r$. By (9), $\|\mathbf{b}_r\|_2 < \frac{g}{m 2^{(m-1)D}}$. Hence, for every $1 \leq j \leq r$,

$$\|\mathbf{b}_j\|_\infty \leq \|\mathbf{b}_j\|_2 \leq \|\mathbf{b}_r\|_2 < \frac{g}{m 2^{(m-1)D}}.$$

Since $\mathbf{b}_j \in \mathcal{L}_0$, Lemma 1 implies $\mathbf{b}_j \cdot \boldsymbol{\pi} = 0$, and therefore $\mathbf{b}_j \in \mathcal{L}_1$ for $1 \leq j \leq r$. By Lemma 2, this yields $p_k \mid (\mathbf{b}_j)_k$ for all $1 \leq j \leq r$, $k \in S$.

Finally, $\text{rank}(\mathcal{L}_1) = r$, while $\text{rank}(\mathcal{L}_0) = r + 1 = m - 2$. Since $\mathbf{b}_1, \dots, \mathbf{b}_{m-2}$ form a basis of $\mathcal{L}_0$ and the first r vectors belong to $\mathcal{L}_1$, the remaining basis vector must satisfy $\mathbf{b}_{m-2} \notin \mathcal{L}_1$.

The Choice for m : Let $G = \lceil \log_2 g \rceil$. By Theorem 5, a sufficient condition for recovering $m - 3$ linearly independent vectors from $\mathcal{L}_1$ is

$$G > (m + 2)D + \frac{m - 3}{2} + \log_2(m \kappa(m)). \quad (10)$$

For the parameterization of the scheme, (4) gives $G \geq 2tb + 3$. Hence, using $D = b + \beta$, a sufficient condition is

$$2tb + 3 > (m + 2)(b + \beta) + \frac{m - 3}{2} + \log_2(m \kappa(m)),$$

or equivalently,

$$2t > (m + 2) \left(1 + \frac{\beta}{b} \right) + \frac{\frac{m-3}{2} + \log_2(m \kappa(m)) - 3}{b}.$$

For small m and large b (for example, $5 \leq m \leq 20$ and $b \geq 15$) $\frac{m-3}{2} + \log_2(m \kappa(m)) - 3 < b$, holds, then a sufficient condition is

$$2t > (m + 2) \left(1 + \frac{\beta}{b} \right).$$

Moreover, when β is small compared with b , this is asymptotically reduced to

$$t > \frac{m + 2}{2}.$$

The choice of m also affects the computational cost of the attack. Indeed, the rank of the publicly computable lattice $\mathcal{L}_0$ is $m - 2$, and hence the cost of LLL

reduction increases with m . Therefore, we seek a relatively small value of m that satisfies the above condition.

For the parameter sets considered in Table 1, $m = 8$ satisfies the condition with a substantial margin. In this case, the rank of $\mathcal{L}_0$ is only 6, while $\text{rank}(\mathcal{L}_1) = 5$, providing a practical balance between the separation condition and the computational cost of lattice reduction.

6 The Cryptanalytic Attack

Section 5 shows that, for a single index set S of size m satisfying (10), LLL reduction of the publicly computable lattice $\mathcal{L}_0$ returns $r = m - 3$ vectors $\mathbf{c} \in \mathcal{L}_1$, each satisfying $p_k \mid c_k$ for every $k \in S$. A single such vector does not determine any p_k . The attack therefore collects many vectors, from many index sets, and combines their divisibility relations. This section describes how, and shows that the combination leaves only a negligible ambiguity, which a short search removes.

Throughout, let $t_{\max} = \max_{i \in [n]} t_i$ and let $\mathcal{W} = [2^b, 2^D)$ be the interval in which key generation confines every secret integers p_i . Both endpoints of $\mathcal{W}$ are determined by the public parameters (n, t, b, β) and are therefore known to the attacker.

To recover the private key integers from these divisibility relations, we need several lattice instances that provide information about the same coordinates. The key idea is to use overlapping index sets: a fixed subset of coordinates is kept common across all instances, while the remaining coordinates are varied. The common coordinates therefore provide multiple divisibility relations for the same private key integers, which can subsequently be combined to eliminate the remaining ambiguity. We formalize this construction using a core and a collection of blocks.

Blocks and the Core: Fix a *core* $\mathcal{C} = \{1, \dots, \kappa\} \subseteq [n]$ with $\kappa < m$ coordinates and put $\tau = m - \kappa$. For $j \in [\ell]$, where $\ell = \lfloor (n - \kappa)/\tau \rfloor$, define the *blocks*

$$S_j = \mathcal{C} \cup \{ \kappa + (j-1)\tau + 1, \dots, \kappa + j\tau \}. \quad (11)$$

Then $|S_j| = m$, every block contains $\mathcal{C}$, and the tails $S_j \setminus \mathcal{C}$ are pairwise disjoint. Applying Theorem 5 to each S_j yields

$$\mathbf{b}^{(j,1)}, \dots, \mathbf{b}^{(j,r)} \in \mathcal{L}_1^{(S_j)}, \quad r = m - 3,$$

and $p_k \mid (\mathbf{b}^{(j,i)})_k$ for every $k \in S_j$.

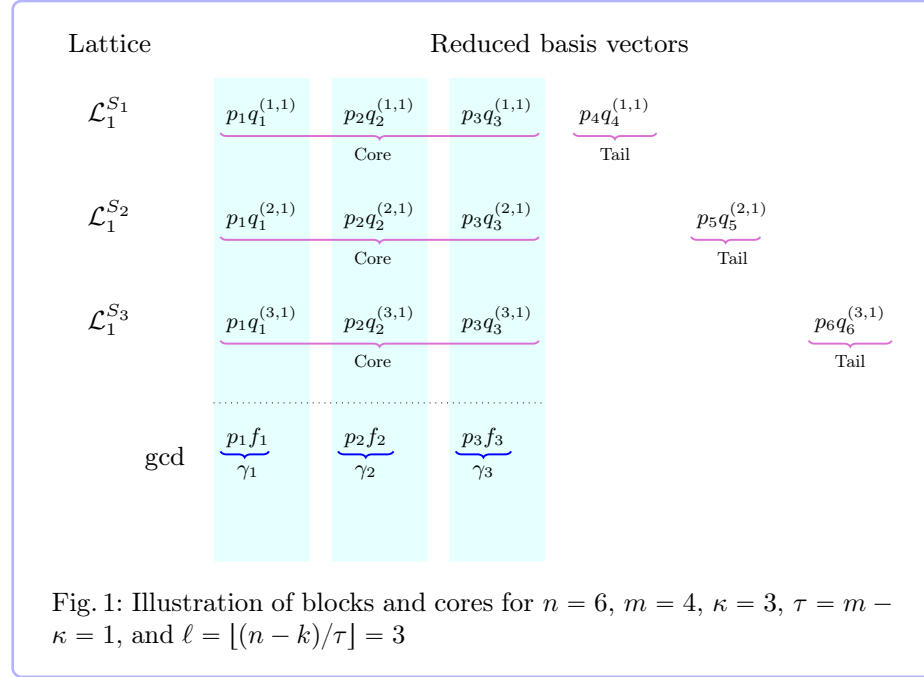
The point of the construction is that the core is common to all blocks. For each $k \in \mathcal{C}$, all ℓr vectors therefore carry a divisibility relation for the *same* secret integer p_k , and we aggregate them by setting

$$\gamma_k = \gcd_{j \in [\ell], i \in [r]} |(\mathbf{b}^{(j,i)})_k|, \quad k \in [\kappa]. \quad (12)$$

Since p_k divides every term, $p_k \mid \gamma_k$, so $\gamma_k = f_k p_k$ for some $f_k \in \mathbb{Z}_{\geq 1}$. Writing $(\mathbf{b}^{(j,i)})_k = p_k q_k^{(j,i)}$ with $q_k^{(j,i)} \in \mathbb{Z}$, the leftover factor is itself a greatest common divisor,

$$f_k = \gcd_{j \in [\ell], i \in [r]} |q_k^{(j,i)}|. \quad (13)$$

Recovering p_k from γ_k thus amounts to removing f_k . For an illustration see Figure 1.



Remark 3. It is important to note that the r vectors $\mathbf{b}^{(j,i)}$ (for a fix j) is obtained from a reduced basis of the same lattice and hence should not be regarded as r independent samples. Accordingly, although ℓr quotients appear in (13), we conservatively regard only ℓ of them as independent contributions, one from each block. These contributions correspond to blocks whose tails are pairwise disjoint.

The Modulus: Fix $1 \leq i \leq j \leq \kappa$, then using (7), we obtain

$$p_i t_i + p_i u - 1 = g k_i, \quad p_j t_j + p_j u - 1 = g k_j \quad (14)$$

Multiplying the relation for i by p_j , the relation for j by p_i , and subtracting eliminates the unknown mask:

$$p_i p_j (t_i - t_j) + (p_i - p_j) = g w_{i,j}, \quad w_{i,j} := p_j k_i - p_i k_j. \quad (15)$$

The left-hand side is computable from the public key once p_i and p_j are known, so each pair of core coordinates yields a multiple of g . Put

$$g_0 = \gcd_{1 \leq i < j \leq \kappa} |p_i p_j (t_i - t_j) + (p_i - p_j)| = c g, \quad c = \gcd_{1 \leq i < j \leq \kappa} |w_{i,j}|, \quad (16)$$

so that recovering g from g_0 amounts to removing the cofactor c , exactly as recovering p_k from γ_k amounts to removing f_k .

Here too the number of quantities exceeds the number of independent ones. A direct computation gives the three-term identity

$$p_1 w_{i,j} = p_i w_{1,j} - p_j w_{1,i}, \quad 1 < i < j \leq \kappa, \quad (17)$$

and we claim that c is already determined by the $\kappa - 1$ quantities $w_{1,2}, \dots, w_{1,\kappa}$. Write $c_\star = \gcd_{2 \leq j \leq \kappa} |w_{1,j}|$. Since these are among the $w_{i,j}$, certainly $c \mid c_\star$. For the converse, $c_\star$ divides both terms on the right of (17), hence $c_\star \mid p_1 w_{i,j}$ for every pair, and so $c_\star \mid p_1 c$. It remains to remove the factor p_1 . Reducing the relation for $i = 1$ in (14) modulo p_1 gives $g k_1 \equiv -1 \pmod{p_1}$, so k_1 is invertible modulo p_1 and $\gcd(k_1, p_1) = 1$; as $w_{1,2} \equiv p_2 k_1 \pmod{p_1}$ and the p_k are pairwise coprime, $\gcd(p_1, w_{1,2}) = 1$ and therefore $\gcd(p_1, c_\star) = 1$. Consequently $c_\star \mid c$, and

$$c = \gcd_{2 \leq j \leq \kappa} |w_{1,j}|. \quad (18)$$

All but $\kappa - 1$ of the $\binom{\kappa}{2}$ quantities in (16) are therefore redundant, and we treat $\kappa - 1$ of them as independent, no more.

Both Cofactors Are Small: It remains to bound f_k and c . We use the following model, which is the only heuristic in the attack: the quotients $q_k^{(j,1)}$ of (13), and likewise the $w_{1,j}$ of (18), are drawn independently and uniformly from $\{1, \dots, M\}$ with $M \rightarrow \infty$. In that limit, the probability of N -tuples of positive integers with greatest common divisor d is $1/(d^N \zeta(N))$ [Nym72, CS87], where ζ is the Riemann zeta function; summing over $d > C$ and comparing with $\int_C^\infty x^{-N} dx$ gives, for any $N \geq 2$,

$$\Pr[\gcd > C] \leq \frac{C^{1-N}}{(N-1) \zeta(N)}. \quad (19)$$

Applying (19) with $N = \ell$ to f_k and with $N = \kappa - 1$ to c , the two cofactor bounds C_p and C_g satisfy

$$\Pr[f_k > C_p] \leq \frac{C_p^{1-\ell}}{(\ell-1) \zeta(\ell)}, \quad \Pr[c > C_g] \leq \frac{C_g^{2-\kappa}}{(\kappa-2) \zeta(\kappa-1)}. \quad (20)$$

Both are finite only when the exponent exceeds 1, which requires $\ell \geq 3$ and $\kappa \geq 4$; we assume both throughout.

With $m = 8$ and $\kappa = 6$ we have $\tau = m - k = 2$, so using every available block gives $\ell = \lfloor (n-6)/2 \rfloor$, which is at least 39 for every parameter set of Table 1 because $n \geq 85$. Substituting $\ell = 39$ and $\kappa = 6$ into (20),

$$\Pr[f_k > 4] \leq 2^{-81}, \quad \Pr[c > 2^{13}] \leq 2^{-54},$$

so the modest choice $C_p = 4$, $C_g = 2^{13}$ already places both failure probabilities below 2^{-50} .

Both bounds are pessimistic in practice. To see how far, we generated 1000 keys for the first parameter set of Table 1, $(n, t, b, \beta) = (85, 22, 60, 1)$, and measured f_k and c directly, taking $m = 8$, $\kappa = 6$. Over the resulting core columns the leftover factor f_k was equal to 1 without exception, so $f_k \leq C_p = 4$ held with three units to spare in every column. The cofactor c took the values 1, 2 and 3 on 970, 27 and 3 keys respectively, so $c \leq C_g = 2^{13}$ held with the cap exceeding the largest observed value by a huge factor. The observed frequency of $c = 1$, namely 97.0%, agrees closely with the $1/\zeta(\kappa - 1) = 96.44\%$ predicted by the above model. So the heuristic is calibrated and not simply conservative.

Algorithm to Recover the private key: Everything needed is now in place. Since $\gamma_k = f_k p_k$ with $f_k \leq C_p$, and since p_k lies in $\mathcal{W}$, the secret integer attached to a core coordinate is recovered by scanning $c_k = 1, 2, \dots, C_p$ and retaining γ_k/c_k whenever c_k divides γ_k and the quotient falls in $\mathcal{W}$. The interval therefore serves as an acceptance test on the quotient rather than as a filter applied to a list of divisors.

A survived candidate tuple $\mathbf{q} = (q_1, \dots, q_\kappa)$ for $(p_1, \dots, p_\kappa)$ determines g_0 through (16), and g is then sought among the quotients $g' = g_0/c$ for those $c \leq C_g$ that divide g_0 . Two tests single it out. The first is immediate: every published t_i is a residue modulo g , so $g > t_{\max}$, and any candidate at or below that threshold is discarded. The second asks g to reproduce the entire public key. Setting

$$u' = (q_1^{-1} \pmod{g'}) - t_1 \pmod{g'},$$

we require $\gcd(t_i + u', g') = 1$, satisfy $(p'_i)^{-1} - u' \equiv t_i \pmod{g'}$, and $p'_i \in \mathcal{W}$ for every $i \in [n]$.

A complete pseudocode of this procedure is provided in Algorithm 1.

Theorem 6. *Let $\text{pk} = (t_1, \dots, t_n)$ be a public key, write $G = \lceil \log_2 g \rceil$, and let $\Lambda = \Theta(nG)$ denote the bit-length of pk . Suppose Algorithm 1 is run with parameters satisfying $m = \mathcal{O}(1)$, $4 \leq \kappa < m$ and $C_p, C_g \leq \text{poly}(\Lambda)$. Then Algorithm 1 runs in time polynomial in the bit-size Λ of the public key.*

Proof. The loop of lines 2–6 runs $\ell = \lfloor (n - \kappa)/\tau \rfloor \leq n$ times. Each iteration computes a basis of the kernel of a $2 \times m$ integer matrix with entries below g , which the Hermite normal form algorithm of Kannan and Bachem [KB79] produces in time polynomial in m and G , with entries of $\mathcal{O}(G)$ bits, and then reduces that basis. Reducing a basis of this size costs $\mathcal{O}(m^6 G^3)$ bit operations [LLL82], and the reduced vectors are no larger. The kernel has rank $m - 2$ and determinant at most $\|\mathbf{1}\|_2 \|\mathbf{t}|_{S_j}\|_2 \leq mg$ by Hadamard’s inequality, so its reduced basis again consists of vectors of $\mathcal{O}(G)$ bits. Line 7 then performs $\mathcal{O}(nm)$ greatest common divisor computations on integers of $\mathcal{O}(G)$ bits.

The loop of lines 8–24 executes at most C_p^κ times and its inner loop at most C_g times, and a single inner iteration performs $\mathcal{O}(n)$ modular inversions modulo an $\mathcal{O}(G)$ -bit integer. Since $\kappa = \mathcal{O}(1)$, the hypothesis $C_p, C_g \leq \text{poly}(\Lambda)$ gives

Algorithm 1: private key recovery by lattice reduction

Input: $\text{pk} = (t_1, \dots, t_n)$ and the public parameters b, β , which determine $\mathcal{W} = [2^b, 2^D)$

Data: Block size m satisfying (10); core size $4 \leq \kappa < m$; cofactor bounds C_p and C_g . For the parameter sets of Table 1 we take $m = 8$, $\kappa = 6$, $C_p = 4$ and $C_g = 2^{13}$

Output: A private key $(p_1, \dots, p_n, g, u)$, or $\perp$

```

1  $\tau \leftarrow m - \kappa$ ;  $r \leftarrow m - 3$ ;  $\ell \leftarrow \lfloor (n - \kappa)/\tau \rfloor$ ;  $t_{\max} \leftarrow \max_{i \in [n]} t_i$ 
2 for  $j = 1$  to  $\ell$  do
3    $S_j \leftarrow \{1, \dots, \kappa\} \cup \{\kappa + (j - 1)\tau + 1, \dots, \kappa + j\tau\}$ 
4    $\mathcal{L}_0^{(j)} \leftarrow \{\mathbf{c} \in \mathbb{Z}^{S_j} : \sum_{k \in S_j} c_k = \sum_{k \in S_j} c_k t_k = 0\}$ 
5    $\mathbf{b}^{(j,1)}, \dots, \mathbf{b}^{(j,r)} \leftarrow$  the  $r$  shortest vectors of an LLL-reduced basis of  $\mathcal{L}_0^{(j)}$ 
6 end
7 for  $k = 1$  to  $\kappa$  do  $\gamma_k \leftarrow \gcd_{j \in [\ell], i \in [r]} |(\mathbf{b}^{(j,i)})_k|$ 
8 foreach  $(c_1, \dots, c_\kappa) \in [C_p]^\kappa$  in increasing order of  $(\max_k c_k, \sum_k c_k)$  do
9   if  $c_k \nmid \gamma_k$  or  $\gamma_k/c_k \notin \mathcal{W}$  for some  $k \in [\kappa]$  then continue
10   $q_k \leftarrow \gamma_k/c_k$  for every  $k \in [\kappa]$ 
11   $g_0 \leftarrow \gcd_{1 \leq i < j \leq \kappa} |q_i q_j (t_i - t_j) + (q_i - q_j)|$ 
12  for  $c = 1$  to  $C_g$  do
13    if  $c \nmid g_0$  then continue
14     $g' \leftarrow g_0/c$ 
15    if  $g' \leq t_{\max}$  or  $\gcd(q_a, g') \neq 1$  for all  $a \in [\kappa]$  then continue
16     $u' \leftarrow (q_1^{-1} \bmod g') - t_1 \bmod g'$ 
17    for  $i = 1$  to  $n$  do
18      if  $\gcd(t_i + u', g') \neq 1$  then go to the next  $c$ 
19       $p'_i \leftarrow (t_i + u')^{-1} \bmod g'$ 
20      if  $p'_i \notin \mathcal{W}$  or  $(p'_i)^{-1} - u' \not\equiv t_i \pmod{g'}$  then go to the next  $c$ 
21    end
22    return  $(p'_1, \dots, p'_n, g', u')$ 
23  end
24 end
25 return  $\perp$ 

```

$C_p^\kappa C_g \leq \text{poly}(\Lambda)$; every iteration count in the algorithm is therefore bounded by n or by a polynomial in Λ , while every arithmetic operation it performs is polynomial in G . Therefore the total cost becomes polynomial in Λ , as claimed. When C_p and C_g are moreover held constant, as they are throughout this paper, the ℓ lattice reductions dominate and the cost is $\mathcal{O}(n m^6 G^3) = \mathcal{O}(\Lambda^3)$.

Remark 4. For the parameter sets in Table 1, we use the default choices $m = 8$, $\kappa = 6$, $C_p = 4$, and $C_g = 2^{13}$, which are not optimized for each individual parameter set. Since, as shown in the next section, all of these parameter sets can be broken within few seconds, optimizing these auxiliary parameters is not necessary for our attack on these parameter sets.

Table 2: Time to recover the private key from the public key alone. Each entry is the maximum recovery time over 100 freshly generated keys, together with the claimed security level for the corresponding parameter set.

n	t	b	β	Claimed security (bits)	Recovery time (s)		n	t	b	β	Claimed security (bits)	Recovery time (s)
85	22	60	1	74	0.23		90	23	60	1	76	0.26
90	26	50	15	65	0.32		95	22	70	1	79	0.31
95	25	55	1	78	0.26		100	20	50	1	78	0.19
100	25	55	1	80	0.28		110	25	60	1	84	0.34
110	30	40	20	64	0.40		125	20	60	1	87	0.30
125	25	40	1	87	0.24		150	20	40	5	87	0.25
150	30	30	20	73	0.49		150	30	50	1	98	0.48
175	30	45	1	105	0.48		175	40	30	20	73	0.76
200	40	70	1	117	1.38		200	50	45	10	99	1.32
225	45	45	5	118	1.54		225	55	40	10	103	1.51
250	60	50	1	132	1.98		250	55	30	20	85	1.67
300	60	50	1	150	2.42		400	50	50	1	156	2.54

7 Impact on the Recommended Parameter Sets

We implemented both the key generation of the cryptosystem and Algorithm 1 in SageMath 10.4, and ran them single-threaded on one core of an Intel Core i9 clocked at 4.1 GHz. Throughout we used the default parameters of Section 6, namely $m = 8$, $\kappa = 6$, $C_p = 4$ and $C_g = 2^{13}$, with every available block.

For each parameter set in Table 1, we generated 100 independent public keys and applied the attack to each instance. In every experiment, the attack successfully recovered the complete private key $(p_1, \dots, p_n, g, u)$, which agreed with the key generated from the key generation. Table 2 reports the maximum recovery time among the 100 experiments for each parameter set, together with its claimed security level.

The claimed security levels range from 64 to 156 bits. In contrast, the maximum observed recovery time is only 2.54 seconds for every parameter set, demonstrating that all the considered instances can be broken within few seconds.

Impossibility of Parameter-Based Countermeasures. It is natural to ask whether a different choice of parameters could prevent the attack. The difficulty is that the attack and the decryption algorithm want the same thing, namely a modulus g that is large compared with the secret integers.

For the attack, this requirement appears through the separation condition. As $\log_2 g$ increases relative to $D = b + \beta$, the useful vectors in $\mathcal{L}_1$ become shorter relative to vectors in $\mathcal{L}_0 \setminus \mathcal{L}_1$, making them easier to isolate using LLL. Taking the smallest usable block size $m = 5$ in (10), the attack succeeds whenever

$$\log_2 g > 7D + 8. \quad (21)$$

Decryption imposes a similar, but substantially stronger, requirement. Correctness requires $g > 4\lambda_{\max}^2$, where $\lambda_{\max}$ is the product of largest t private key integers, each of at most D bits. Consequently, the modulus must satisfy

$$\log_2 g \gtrsim 2tD. \quad (22)$$

Since $t \geq 20$ for every proposed parameter set, (22) is substantially stronger than the attack condition (21).

Thus, avoiding the attack while maintaining decryption correctness would require reducing $2tD$ below approximately $7D$, which would force $t \leq 3$. Such a choice, however, leads to a different vulnerability: a message with at most three nonzero positions can be recovered by enumerating its possible supports, of which there are $\binom{n}{3} = \mathcal{O}(n^3)$, placing the attack within the combinatorial regime considered by the authors [GTLN26]. The remaining parameter adjustments do not resolve this tension. Increasing g only strengthens the separation exploited by the attack, while increasing D through a wider parameter window also increases the required modulus size, which scales as $2tD$. The weakness therefore lies in the design of the scheme rather than in the choice of its parameters.

8 Conclusion

We have shown that the knapsack McEliece-based cryptosystem of Gómez-Torrecillas *et al.* does not provide the security claimed for it. From the public key alone, our attack recovers the complete private key in time polynomial in the bit-length of the public key. In practice, we recover the private key for all twenty-four proposed parameter sets in under 3 seconds on a single CPU core, despite claimed security levels of up to 156 bits. The attack requires neither a ciphertext nor any additional information about the private key.

The attack exposes a structural weakness in the construction. The correctness requirement for decryption forces the modulus to be sufficiently large, and this same condition creates the separation that allows LLL to recover the short vectors carrying the private key relations. Combining these relations across overlapping blocks then recovers the complete private key. Thus, the failure is not due to an unfortunate choice of parameters, but follows from the interaction between the decryption condition and the structure of the public key. These results demonstrate that the proposed cryptosystem is fundamentally vulnerable to public-key-only key recovery.

References

- Bri84. Ernest F Brickell. Breaking iterated knapsacks. In *Workshop on the Theory and Application of Cryptographic Techniques*, pages 342–358. Springer, 1984.
- CC71. John William Scott Cassels and John William Scott Cassels. *An introduction to the geometry of numbers*, volume 99. Springer, 1971.
- CG20. Jean-Sébastien Coron and Agnese Gini. A polynomial-time algorithm for solving the hidden subset sum problem. In *Annual International Cryptology Conference*, pages 3–31. Springer, 2020.
- CJL⁺92. Matthijs J Coster, Antoine Joux, Brian A LaMacchia, Andrew M Odlyzko, Claus-Peter Schnorr, and Jacques Stern. Improved low-density subset sum algorithms. *Computational complexity*, 2(2):111–128, 1992.

- CS87. J. Chidambaraswamy and R. Sitaramachandrarao. On the probability that the values of m polynomials have a given g.c.d. *Journal of Number Theory*, 26(3):237–245, 1987.
- DJ24. Dipayan Das and Antoine Joux. Key recovery attack on the partial vandermonde knapsack problem. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 205–225. Springer, 2024.
- GTLN26. José Gómez-Torrecillas, FJ Lobillo, and Gabriel Navarro. A knapsack mceliece-based public key cryptosystem: J. gómez-torrecillas et al. *Designs, Codes and Cryptography*, 94(8):172, 2026.
- KB79. Ravindran Kannan and Achim Bachem. Polynomial algorithms for computing the smith and hermite normal forms of an integer matrix. *siam Journal on Computing*, 8(4):499–507, 1979.
- LLL82. Arjen K. Lenstra, Hendrik W. Lenstra, and László Lovász. Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534, 1982.
- LO85. Jeffrey C Lagarias and Andrew M Odlyzko. Solving low-density subset sum problems. *Journal of the ACM (JACM)*, 32(1):229–246, 1985.
- McE78. Robert J McEliece. A public-key cryptosystem based on algebraic. *Coding Thv*, 4244(1978):114–116, 1978.
- MH78. Ralph Merkle and Martin Hellman. Hiding information and signatures in trapdoor knapsacks. *IEEE transactions on Information Theory*, 24(5):525–530, 1978.
- NS97. Phong Nguyen and Jacques Stern. Merkle-hellman revisited: a cryptanalysis of the qu-vanstone cryptosystem based on group factorizations. In *Annual International Cryptology Conference*, pages 198–212. Springer, 1997.
- NS99. Phong Nguyen and Jacques Stern. The hardness of the hidden subset sum problem and its cryptographic implications. In *Annual International Cryptology Conference*, pages 31–46. Springer, 1999.
- Nym72. James E Nymann. On the probability that k positive integers are relatively prime. *Journal of number theory*, 4(5):469–473, 1972.
- Odl90. Andrew M Odlyzko. The rise and fall of knapsack cryptosystems. *Cryptology and computational number theory*, 42(2), 1990.
- RSA78. Ronald L Rivest, Adi Shamir, and Len Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.
- Sha82. Adi Shamir. A polynomial time algorithm for breaking the basic merkle-hellman cryptosystem. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 145–152. IEEE, 1982.